

Homework 2

Due by: June 29, 23:59

General Guidelines:

1. **Watch the online tutorial for introduction to this homework (located in the Moodle)**
2. Don't use any external libraries for this homework, except native C++ libs.
3. You should use only C++11 for exercises 2-3. You may use C++20 for exercise 1.
4. Make sure to create new classes and files with the exact names defined ahead. Otherwise, you will face compilation errors during automatic testing.
5. The homework assignment is 100% auto graded. You may resubmit your solution to gradescope multiple times before the due date of this homework assignment.
6. This homework assignment includes several .h files. Read ahead.
7. The automatic tests for this homework can run **few minutes**, per submission.
8. In total, you should upload (together) these files to gradescope:
 - `prefix_sum_2D_impl.h` (exercise #1)
 - `tree_barrier_impl.h` (exercise #2)
 - `hlock_impl.h` (exercise #3)

Exercise 1: 2D Prefix Sum – 30 points

Implement the 2D prefix sum algorithm in C++. Given a 2D table A (of size $N \times N$), compute $A[i,j] = \sum_{m=0}^i \sum_{n=0}^j A[m,n]$. Provide three implementations: a serial implementation, a multithreaded implementation, and a parallel implementation using the Intel TBB library.

Create the `prefixsum_2D_impl.h` file with a sequential and two versions of parallel implementations to the 2D prefix sum problem. These implementations should comply with the header file (you should `#include` it) and the exact complexity requirements defined in the `prefix_sum_2d.h` header file.

Implementations should update the table in-place with the prefix sum values.

Please read carefully the documentation in `prefix_sum_2d.h`

You should use at least C++17 for TBB (if you run your code locally). Using C++20 is also OK for this specific exercise. Gradescope will compile your code with a C++20 compiler version.

Upload `prefix_sum_2D_impl.h` file with your implementations to gradescope.

Exercise 2: a combining binary tree barrier – 35 points

The implementation of sense barriers discussed in class may suffer from high-memory contention. This is because a *counter* shared variable, updated by all threads entering the barrier. One way to reduce this contention (possibly at the cost of increased latency) is to use a combining paradigm like a combining tree discussed in class.

Implement a combining **binary** tree barrier, `BinaryTreeBarrier`, class which inherits the abstract class in `tree_barrier.h` file.

You can assume that the input parameter of number of threads to a binary tree barrier object is **always a power of 2**.

TIP (not mandatory): use the original code of a sense barrier (shared in class) for the nodes of the combining tree barrier. Modify this original code as needed to create the logic of a combining tree barrier.

Upload *tree_barrier_impl.h* file with your binary tree barrier implementation to gradescope.

Exercise 3: Hierarchical locks – 35 points

Implement a hierarchical lock. A hierarchical lock enforces ordering of locks and unlock ops to prevent deadlocks. A hierarchical lock is basically a mutex with the following differences:

- Every hierarchical lock has a unique level (aka id)
- The lock op should fail if the same thread already locked another hierarchical lock with higher (or equal) level.
- Upon a failure to acquire a lock, the *lock* op should throw an exception object of the *HierarchicalMutexException* class (defined in *hlock.h*). Use the C++ “throw” mechanism.
- Unlock operations by the same thread should be applied in the exact reversed order of their corresponding lock ops (otherwise, some future lock ops may throw the runtime errors). You can assume this usage pattern. No need to enforce it in your implementation.

You should develop a hierarchal lock class “*HierarchicalMutex_impl*” which is a subclass of the *HierarchicalMutex* base class (defined in *hlock.h*).

You can use *std::mutex* for the basic lock/unlock of a hierarchical lock

Upload *hlock_impl.h* file with your implementation of *HierarchicalMutex_impl* class.

Given

```
HierarchicalMutex_impl mutexA(1000);  
HierarchicalMutex_impl mutexB(10000);
```

A valid use case (assume one thread):

```
mutexA.lock();  
mutexB.lock();  
mutexB.unlock();  
mutexA.unlock();
```

An invalid use case (assume two threads):

Thread 1	Thread2
<code>mutexA.lock();</code>	<code>mutexB.lock();</code>
<code>mutexB.lock();</code>	<code>mutexA.lock(); // should throw a runtime error</code>

```
mutexB.unlock();
```

```
mutexA.unlock();
```

```
mutexA.unlock();
```

```
mutexB.unlock();
```

Good luck!